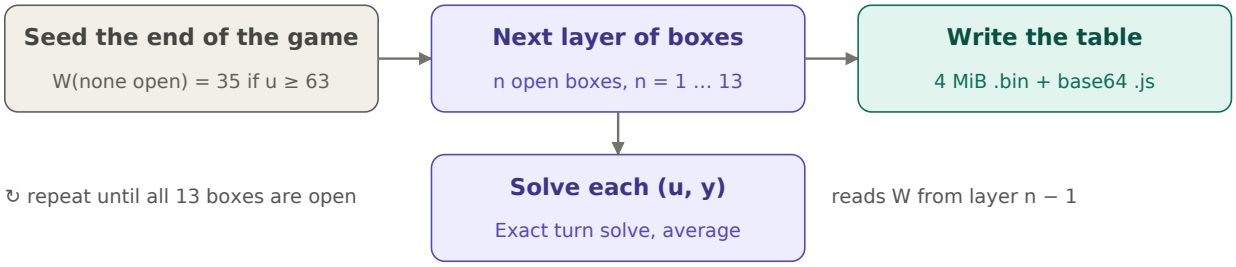
Perfect play in the Yahtzee engine

How the exact table works, from the updated repo (slayyyyer/yahtzee): src/engine.js, tools/solve-optimal.js, docs/plans/2026-10-27-optimal-table.md

The computer used to solve each turn exactly but guess the value of the boxes left afterward. That guess is now gone. An offline generator computes the exact value of every reachable game state once, ships it as a 4 MiB table, and the engine looks values up instead of estimating them. Under this game's rules the empty card is worth **254.143784** expected points.

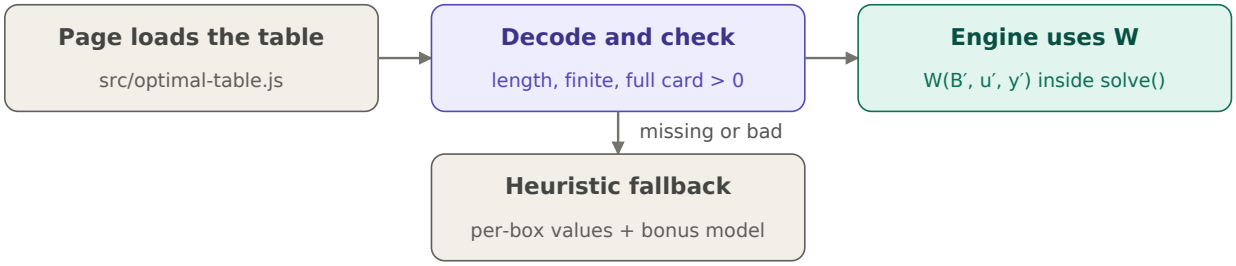
A correction to my earlier PDF: I gave 254.59 as the perfect-play target. That number is the official-rules benchmark. This repo deliberately uses a looser extra-Yahtzee rule, so its own exact optimum is 254.14.

1. Generating the table (offline, about 1.5 minutes)



Each layer only needs the layer below it, so the generator walks upward by number of open boxes. Every (u, y) in a layer is solved with the same turn solver the game uses, and the result is the average over the 252 possible opening rolls.

2. Using the table (in the browser)



The table loads through a plain script tag, so double-clicking index.html still works with no fetch. If the table is missing, the wrong length, or fails a sanity check, the engine quietly falls back to the old heuristic.

What changed in the code

- **engine.js**: new `solveTurnValues` (a no-allocation turn solver for the generator), `decodeOptimalTable`, `tableIndex`, and an `optimalTable` option on `createEngine`. The fallback's last-box value now includes the upper bonus.
- **tools/solve-optimal.js**: the generator, with `--official`, `--strict`, `--workers` and `--max-pop` modes.
- **Outputs**: tools/optimal-table.bin (4,194,304 bytes) and src/optimal-table.js (5.6 MB base64).
- **Wiring**: index.html and app.js load and pass the table; train.js gets `--optimal`; test-engine.js grows from 36 to 50 checks.

The math

Backward induction over the number of open boxes, with an exact turn solve inside each state.

State and index

B is the 13-bit mask of open boxes, u the upper total capped at 63, and y is 1 only when the Yahtzee box is closed with 50. W(B, u, y) is the expected additional score from the start of a turn. State (B, u, y) lives at index i in a Float32 array of 1,048,576 entries.

$$i(B, u, y) = (64\,B + u) \cdot 2 + y$$

Not every index is a real state: y = 1 needs the Yahtzee box closed. Counting the valid combinations gives the 786,432 states the README quotes:

$$(4096 \cdot 1 + 4096 \cdot 2) \cdot 64 = 12288 \cdot 64 = 786432$$

Terminal values and the turn recurrence

With no boxes left, only the upper bonus remains:

$$W(\varnothing, u, y) = 35 \cdot [\, u \geq 63 \,]$$

Banking a final roll s in box c is worth its points, plus 100 for an extra Yahtzee (five of a kind while y = 1), plus the table value of what remains. Here u′ adds the points to u when c is an upper box (capped at 63), and y′ becomes 1 only if c is the Yahtzee box and scored 50.

$$T_c(s) = \text{score}_c(s) + 100 \cdot [\,\text{five}(s) \wedge y = 1\,] + W(B \setminus \{c\},\, u',\, y')$$

The two rerolls then work backward exactly as before, and the state's value averages over the opening roll:

$$V_0(s) = \max_{c \in B} T_c(s), \qquad V_r(s) = \max_{k \subseteq s} \sum_e P(e)\, V_{r-1}(k + e), \quad r = 1, 2$$

$$W(B, u, y) = \sum_s P(s)\, V_2(s) \qquad (252 \text{ fresh rolls})$$

Three rule sets, three exact values

Mode	Extra Yahtzee placement	W(full card)
Repo rules (shipped)	Any open box, +100, no forced placement	254.143784
--official	Matching upper box first; else any open lower box (25 / 30 / 40 or dice sum); an open upper box may be zeroed	254.589097
--strict	Same, but zeroing an upper box only after the lower section is full	254.587729

The official mode reproduces the published 254.5896 benchmark to within 0.0005, which is the independent proof that the recurrence and indexing are right. Neither validation mode writes the shipped files.

Results and how to reproduce

Self-play over 5,000 seeded games, exact table versus the old heuristic.

Metric	Exact table	Old heuristic
Mean final score	254.16	250.39
Standard deviation	58.84	61.15
Upper bonus earned	67.6% of games	64.2% of games
Yahtzee bonus	9.38 points per game	not reported
Exact expected value	254.143784	not applicable

The table's mean of 254.16 sits inside its 95% confidence interval (±1.63) of the exact 254.143784, so play matches the math. The gain over the heuristic is about 3.8 points per game, or roughly 1.5%.

What I checked

- Ran `node tools/test-engine.js` on the uploaded repo: all **50 checks pass**.
- Read the shipped table directly: the full-card value is 254.143784, the Chance-only value is 23.33 (its known solo expectation), and 786,306 entries are non-zero. The 126 zero entries are exactly the terminal states with fewer than 63 upper points, as expected.
- I did **not** regenerate the table or rerun self-play. The generation time, official and strict values, and self-play figures come from the repo's plan and README.

Reproduce it

```
node tools/test-engine.js                          # 50 checks
node tools/solve-optimal.js --official --workers=16   # benchmark, no files
node tools/solve-optimal.js --workers=16           # regenerate shipped table
node tools/train.js --games=5000 --optimal         # self-play
```

Limits worth knowing

- **Expected score, not win probability.** The table maximizes points. Against a human the strategy that maximizes the chance of winning can differ (for example when trailing late), and that two-player problem is not solved exactly.
- **Repo rules only.** The table encodes this game's permissive joker rule. Changing scoring rules means regenerating it.
- **Page weight.** The base64 script is 5.6 MB, which every page load must download and decode once.
- **Float32 storage.** Values are stored in single precision (about 7 digits), which is far finer than any decision needs.